

```
In [ ]: # Initialize Otter
import otter
grader = otter.Notebook("practice01.ipynb")
```

Student names and e-mails:

YOUR NAME — *your@calvin.edu*

YOUR NAME — *your@calvin.edu*

Practice 01 — DataFrame Basics

In this practice you will work with a real dataset about forest fires in Brazil. Each task is tagged with the SLO it covers:

SLO	Description
02A	Manipulate the structure and contents of pandas DataFrames
02B	Sort, filter, and query DataFrames
02C	Choose appropriate visual encodings in a visualization

The Dataset: Amazon Forest Fires (Brazil, 1998–2017)



This dataset reports monthly counts of forest fires by state, recorded by Brazil's national space research agency INPE. Each row represents one state–month–year combination.

```
In [ ]: import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
```

The cell below loads the data from a CSV file into a pandas DataFrame called `fires`. Run it and look at the first few rows.

```
In [ ]: fires = pd.read_csv(
    'https://cs.calvin.edu/courses/data/202/26fa/datasets/amazon.csv',
    encoding='latin-1'
)
fires.head()
```

Notice that the `month` column contains **Portuguese names** (e.g., `Janeiro`, `Fevereiro`). The cell below translates them to integers (1–12) and corrects the `date` column to match. Run it — you don't need to modify it.

```
In [ ]: month_map = {
        'Janeiro': 1, 'Fevereiro': 2, 'Marco': 3, 'Abril': 4,
        'Maio': 5, 'Junho': 6, 'Julho': 7, 'Agosto': 8,
        'Setembro': 9, 'Outubro': 10, 'Novembro': 11, 'Dezembro': 12
    }
fires['month'] = fires['month'].map(month_map)
fires['date'] = pd.to_datetime(fires[['year', 'month']].assign(day=1))
fires.head()
```

Part 1 — Understanding the DataFrame

Before any analysis, explore what you have. Key tools:

Expression	What it gives you
<code>df.shape</code>	Tuple (rows, columns)
<code>df.columns</code>	Index of column names
<code>df['col']</code>	A Series (one column)
<code>df['col'].sum()</code>	Total of that column
<code>df['col'].idxmax()</code>	Index of the max value
<code>df.loc[idx, 'col']</code>	Value at row <code>idx</code> , column <code>'col'</code>

```
In [ ]: fires.info()
```

```
In [ ]: fires.describe()
```

Task 02A.1 — Shape and Columns (1 pt)

Using `fires.shape`, assign:

- `n_rows` to the number of rows
- `n_cols` to the number of columns

Use the attribute — don't type the literal numbers.

```
In [ ]: n_rows = fires.shape[0] # SOLUTION
        n_cols = fires.shape[1] # SOLUTION
        print(f'Rows: {n_rows} | Columns: {n_cols}')
```

```
In [ ]: grader.check("02A.1")
```

Task 02A.2 — Accessing Columns and Computing (2 pts)

Using the `number` column (fires per record) and the `state` column:

1. Assign the **total** number of fires across all rows to `total_fires`.
2. Find the **state that had the single highest monthly fire count** in the dataset. Assign its name (a string) to `peak_state`.

Hints: `.sum()`, `.idxmax()`, and `.loc[]` will be useful.

```
In [ ]: total_fires = fires['number'].sum() # SOLUTION
        peak_idx = fires['number'].idxmax() # SOLUTION
        peak_state = fires.loc[peak_idx, 'state'] # SOLUTION
        print(f'Total fires across all records: {total_fires:.0f}')
        print(f'State with highest single-month count: {peak_state}')
```

```
In [ ]: grader.check("02A.2")
```

Task 02A.3 — Adding a Column (2 pts)

Add a new column called `'decade'` to `fires` that records the **decade** of each row as a string:

- 1990–1999 → `'1990s'`
- 2000–2009 → `'2000s'`
- 2010–2019 → `'2010s'`

Hint: integer division (`//`) on the `year` column gets you the decade start; string concatenation adds the `'s'`.

```
In [ ]: fires['decade'] = (fires['year'] // 10 * 10).astype(str) + 's' # SOLUTION
        fires[['year', 'decade']].drop_duplicates().sort_values('year').head(10)
```

```
In [ ]: grader.check("02A.3")
```

Part 2 — Sorting and Filtering

Pandas lets you focus on subsets of your data:

Operation	Syntax	Example
Boolean filter	<code>df[condition]</code>	<code>df[df['year'] > 2010]</code>
Multiple conditions	<code>df[cond1 & cond2]</code>	Use <code>&</code> not and
Filter by list	<code>df['col'].isin(lst)</code>	checks membership
Sort	<code>df.sort_values('col', ascending=False)</code>	largest first
Top N	<code>.head(N)</code> after sort	

```
In [ ]: # Records from 2015 onward, during August (peak fire season)
late_august = fires[(fires['year'] >= 2015) & (fires['month'] == 8)]
print(f'{len(late_august)} rows matched')
late_august.sort_values('number', ascending=False).head()
```

Task 02B.1 — Filtering (2 pts)

Filter `fires` to include only rows for the state of **'Mato Grosso'**. Assign the result to `mato_grosso`.

Then print the number of rows in your filtered dataframe.

```
In [ ]: mato_grosso = fires[fires['state'] == 'Mato Grosso'] # SOLUTION
print(f'Mato Grosso rows: {len(mato_grosso)}')
mato_grosso.head()
```

```
In [ ]: grader.check("02B.1")
```

Task 02B.2 — Sorting (2 pts)

From the full `fires` dataframe, find the **5 rows with the highest fire counts**. Assign the result to `top5`.

Then display only the `state`, `year`, `month`, and `number` columns of `top5`.

```
In [ ]: top5 = fires.sort_values('number', ascending=False).head(5) # SOLUTION
top5[['state', 'year', 'month', 'number']]
```

```
In [ ]: grader.check("02B.2")
```

Task 02B.3 — Compound Filtering (2 pts)

The north region of Brazil (the Amazon basin) includes these states:

```
north_states = ['Acre', 'Amapa', 'Amazonas', 'Para', 'Rondonia', 'Roraima', 'Tocantins']
```

Create a new dataframe `north_recent` containing only rows where **both** conditions hold:

1. The state is in `north_states`
2. The year is **2010 or later**

Print its shape.

```
In [ ]: north_states = ['Acre', 'Amapa', 'Amazonas', 'Para', 'Rondonia', 'Roraima', 'Tocantins']
north_recent = fires[fires['state'].isin(north_states) & (fires['year'] >= 2010)] # SOLUTION
print(f'north_recent shape: {north_recent.shape}')
north_recent.head()
```

```
In [ ]: grader.check("02B.3")
```

Part 3 — Visual Encodings

A chart maps data to **visual properties** called **encodings**:

Encoding	Plotly Express argument	Best for
x-axis	x=	time, ordered categories
y-axis	y=	numeric values
color	color=	categories or continuous gradient
size	size=	quantity (use carefully)
facet	facet_col=	small multiples by category

```
In [ ]: fires_per_year = fires.groupby('year')['number'].sum().reset_index()

fig_example = px.bar(
    fires_per_year,
    x='year',
    y='number',
    title='Total Forest Fires in Brazil per Year',
    labels={'number': 'Number of Fires', 'year': 'Year'}
)
fig_example.show()
```

Task 02C.1 — Line Plot for One State (2 pts)

Using your `mato_grosso` dataframe from Task 02B.1, create a **line plot** showing fire counts over time:

- x-axis: 'date'
- y-axis: 'number'
- A descriptive title
- Axis labels via the `labels=` argument

Assign the figure to `fig1` and display it.

```
In [ ]: # BEGIN SOLUTION
fig1 = px.line(
    mato_grosso,
    x='date',
    y='number',
    title='Forest Fires in Mato Grosso Over Time',
    labels={'number': 'Number of Fires', 'date': 'Date'}
)
# END SOLUTION
fig1.show()
```

Task 02C.2 — Comparing Multiple States (3 pts)

One state is interesting, but comparison reveals the bigger picture. Using your `north_recent` dataframe (Task 02B.3), create a line plot comparing fire trends across all north region states.

Requirements:

- Chart type: **line**
- x-axis: `'date'`
- y-axis: `'number'`
- **Color** encoding: `'state'`
- A title and axis labels

Assign to `fig2` and display it.

```
In [ ]: # BEGIN SOLUTION
fig2 = px.line(
    north_recent,
    x='date',
    y='number',
    color='state',
    title='Forest Fires in Northern Brazil (2010-2017)',
    labels={'number': 'Number of Fires', 'date': 'Date', 'state': 'State'}
)
# END SOLUTION
fig2.show()
```

Task 02C.3 — Evaluate Your Visualization (2 pts)

Look critically at `fig2`. In **3–5 sentences**, answer:

1. Does the color encoding help you compare states? Why or why not?
2. What would you change to make the patterns clearer?
3. What does this plot **not** show — what information is hidden or lost?

Edit the cell below and write your answer.

Your answer here.

Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

Submit the `.zip` file to our Moodle assignment page.

```
In [ ]: # Save your notebook first, then run this cell to export your submission.
grader.export(run_tests=True)
```

